

CSP Parser & Evaluator

Pipeline de Verificación y Propagación de Restricciones

Proyecto CSP Parser

2026

Contents

Introducción	4
¿Qué es CSP Parser & Evaluator?	4
Características Principales	4
Filosofía de Diseño	4
Tipos de Datos	5
Variables Soportadas	5
Tipo Boolean	5
Tipo Set	5
Tipo Integer	5
Tipo Numeric (Real)	6
Operadores	7
Operadores Aritméticos	7
Operadores Relacionales	7
Operadores Lógicos	7
Operadores de Conjuntos	7
Funciones	9
Funciones Builtin	9
Matemáticas Básicas	9
Trigonométricas	9
Exponenciales	9
Especiales	9
Funciones User-Defined	9
Declaración	10
Verificación (FunctionChecker)	10
Implementación en C	10
Runtime	10
Arquitectura del Pipeline	12
Flujo de Datos	12
Programas del Pipeline	12
1. SyntaxChecker	12

2. JsonToGraph	13
3. FunctionChecker	13
4. CSPEval	13
5. ForwardChain	14
6. FwdConsistency	14
7. BwdConsistency	14
Compilación e Instalación	15
Requisitos	15
Instalación de Dependencias	15
Compilación	15
Verificación	15
Limpieza	15
Uso	16
Pipeline Completo	16
Programas Individuales	16
Validar Sintaxis	16
Generar Hipergrafo	16
Verificar Funciones	16
Propagar Dominios	16
Formato JSON de Entrada	16
Estructura Básica	16
Variables	17
Expresiones	17
Funciones (Opcional)	17
Ejemplo Completo	17
Grid Bitmask (Variables Numeric)	19
Motivación	19
Estructura	19
Nivel 1: Interval	19
Nivel 2: Grid 20-pt	19
Nivel 3: Bitmask	19
Operaciones de Narrowing	19
Sincronización con Active[]	20
Ventajas	20
Desventajas	20
Alternativas Consideradas	20
8 Reglas Semánticas (SyntaxChecker)	21
Regla 1: Variables Declaradas Sin Usar	21
Regla 2: Funciones Declaradas Sin Usar	21
Regla 3: Variables Usadas Sin Declarar	21
Regla 4: Funciones Usadas Sin Declarar	22
Regla 5: Redefinición de Variable	22
Regla 6: Redefinición de Función	22
Regla 7: Tipo Incorrecto en Expresión	23

Regla 8: Override de Builtin	23
Integración con GecodeInfo	24
Análisis del Espacio de Soluciones	24
Flujo de Trabajo	24
Uso	24
Salida de GecodeInfo	24
Bug Crítico Resuelto	25
Variables Numéricas: Rescalado a Malla	25
Ejemplos	26
Ejemplo 1: Sistema Simple	26
Ejemplo 2: Conjuntos	26
Ejemplo 3: Función User-Defined	26
Conclusiones	28
Ventajas del Sistema	28
Casos de Uso	28
Próximos Pasos	28
Referencias	28

Introducción

¿Qué es CSP Parser & Evaluator?

CSP Parser & Evaluator es un pipeline modular de verificación, evaluación y propagación de restricciones para problemas de **Constraint Satisfaction (CSP)**. Diseñado con filosofía Unix: programas pequeños, cada uno hace una cosa, se pueden encadenar.

El sistema funciona como un pipeline que:

1. **Valida** sintaxis y semántica (8 reglas)
2. **Construye** hipergrafos de restricciones desde JSON
3. **Verifica** existencia de funciones user-defined (.o/.so)
4. **Propaga** dominios mediante AC-3, forward/backward consistency
5. **Retorna** resultados en formato JSON

Características Principales

- [OK] **Validación Semántica:** 8 reglas que garantizan consistencia
- [OK] **Hipergrafo CSP:** Representación eficiente de restricciones
- [OK] **4 Tipos de Variables:** boolean, set, integer, numeric
- [OK] **Grid Bitmask 20-pt:** Discretización eficiente de dominios numéricos
- [OK] **Aritmética de Intervalos:** IvMake, IvAdd, IvMul, IvDiv, IvContains, etc.
- [OK] **Funciones Builtin:** abs, sqrt, sin, cos, ln, exp, interval, complex
- [OK] **Funciones User-Defined:** Carga dinámica de .o/.so
- [OK] **4 Motores de Propagación:** AC-3, forward/backward consistency, forward chaining
- [OK] **Formato JSON:** Pipeline componible vía stdin/stdout
- [OK] **9 Programas Standalone:** Cada uno < 300 KB

Filosofía de Diseño

“Unix se diseñó con 3 principios: Programas pequeños, Cada uno hace una cosa, Se pueden encadenar. Eso es pipeline.”

Este proyecto encarna esos principios:

Principio	Implementación
Programas pequeños	9 ejecutables standalone, cada uno < 300 KB
Cada uno hace una cosa	SyntaxChecker solo valida, JsonToGraph solo convierte, etc.
Se pueden encadenar	Pipeline vía JSON (stdout -> stdin), componible

Tipos de Datos

Variables Soportadas

El sistema soporta 4 tipos de variables:

Tipo	Kind	Dominio	Valor	Storage
boolean	vkBoolean	[true, false]	subconjunto	TLabeledSet
set	vkSet	["A", "B", "C", ...]	subconjunto	TLabeledSet
integer	vkInteger	lista explícita [1,5,10,...]	subconjunto	TLabeledSet
numeric	vkNumeric	[lo, hi]	[lo', hi'] subset [lo, hi]	Interval + Grid

Tipo Boolean

Dominio: [true, false] **Valor:** Subconjunto de {true, false}

Ejemplos:

```
{"name": "activo", "type": "boolean", "domain": [true, false], "value": [true]}
{"name": "flag", "type": "boolean", "domain": [true, false], "value": [true, false]}
```

Tipo Set

Dominio: Lista de labels (strings) **Valor:** Subconjunto de labels

IMPORTANTE: Labels en expresiones son identificadores sin comillas:

```
{
  "name": "estado",
  "type": "set",
  "domain": ["ALTA", "MEDIA", "BAJA"],
  "value": ["ALTA", "MEDIA"]
}
```

Uso en expresión:

```
{"constraints": ["estado IN {ALTA, MEDIA}"]}
```

(No usar comillas: "estado IN {"ALTA", "MEDIA"})

Tipo Integer

Dominio: Lista EXPLÍCITA de enteros válidos (no rango) **Valor:** Subconjunto de dominio

```
{
  "name": "nivel",
  "type": "integer",
  "domain": [1, 5, 10, 20, 50, 100],
  "value": [10, 20, 50]
}
```

Storage: Internamente usa `TLabelledSet` con `labels = strings` de enteros.

Tipo Numeric (Real)

Dominio: Intervalo continuo $[lo, hi]$ **Valor:** Subintervalo $[lo', hi']$ con $lo \leq lo' \leq hi' \leq hi$

```
{  
  "name": "temperatura",  
  "type": "numeric",  
  "domain": [0.0, 100.0],  
  "value": [20.0, 25.0]  
}
```

Storage: Representación híbrida:

1. **Interval:** `Lo, Hi` (bounds del dominio)
2. **Grid 20-pt:** 21 puntos discretos `GridPts[0..20]`
3. **Bitmask:** `GridMask` (32 bits, usa 21) para narrowing rápido

Operadores

Operadores Aritméticos

Aplicables a tipos `integer` y `numeric`:

Operador	Descripción	Ejemplo
+	Suma	<code>x + y</code>
-	Resta	<code>x - y</code>
*	Multiplicación	<code>x * y</code>
/	División	<code>x / y</code>

Operadores Relacionales

Comparación de valores:

Operador	Descripción	Ejemplo
=	Igual	<code>x = 10</code>
<>	Diferente	<code>x <> 0</code>
<	Menor que	<code>x < y</code>
>	Mayor que	<code>x > y</code>
<=	Menor o igual	<code>x <= 100</code>
>=	Mayor o igual	<code>x >= 0</code>

Operadores Lógicos

Aplicables a tipo `boolean`:

Operador	Descripción	Ejemplo
AND	Conjunción	<code>activo AND (nivel > 5)</code>
OR	Disyunción	<code>error OR warning</code>
NOT	Negación	<code>NOT activo</code>
IMPLICA	Implicación	<code>activo IMPLICA (nivel > 0)</code>

Operadores de Conjuntos

Aplicables a tipo `set`:

Operador	Descripción	Ejemplo
IN	Pertenencia	<code>estado IN {ALTA, MEDIA}</code>
UNION	Unión	<code>A UNION B</code>
INTERSECT	Intersección	<code>A INTERSECT B</code>
DIFFERENCE	Diferencia	<code>A DIFFERENCE B</code>
SUBSET	Subconjunto	<code>A SUBSET B</code>
CARDINALITY	Tamaño	<code>CARDINALITY(A)</code>

Ejemplo completo:

```
{
  "variables": [
    {"name": "A", "type": "set", "domain": ["X", "Y", "Z"], "value": ["X", "Y"]},
    {"name": "B", "type": "set", "domain": ["X", "Y", "Z"], "value": ["Y", "Z"]}
  ],
  "expressions": [
    {"constraints": ["A UNION B = {X, Y, Z}"]},
    {"constraints": ["A INTERSECT B = {Y}"]},
    {"constraints": ["CARDINALITY(A) = 2"]}
  ]
}
```


Funciones

Funciones Builtin

Funciones pre-registradas que **NO se declaran** en JSON "functions":

Matemáticas Básicas

Función	Descripción	Ejemplo
<code>abs(x)</code>	Valor absoluto	<code>abs(-5) = 5</code>
<code>sqrt(x)</code>	Raíz cuadrada	<code>sqrt(16) = 4</code>
<code>sqr(x)</code>	Cuadrado	<code>sqr(3) = 9</code>

Trigonométricas

Función	Descripción	Rango
<code>sin(x)</code>	Seno	$[-1, 1]$
<code>cos(x)</code>	Coseno	$[-1, 1]$
<code>tan(x)</code>	Tangente	$\mathbb{R}$
<code>asin(x)</code>	Arcoseno	$[-\pi/2, \pi/2]$
<code>acos(x)</code>	Arcocoseno	$[0, \pi]$
<code>atan(x)</code>	Arcotangente	$(-\pi/2, \pi/2)$

Exponenciales

Función	Descripción	Ejemplo
<code>exp(x)</code>	e^x	<code>exp(1) ~= 2.718</code>
<code>ln(x)</code>	Logaritmo natural	<code>ln(e) = 1</code>
<code>log(x)</code>	Logaritmo base 10	<code>log(100) = 2</code>
<code>pow(x, y)</code>	Potencia x^y	<code>pow(2, 3) = 8</code>

Especiales

Función	Descripción	Tipo de retorno
<code>interval(lo, hi)</code>	Crear intervalo numerico	numeric
<code>complex(re, im)</code>	Crear número complejo	complex
<code>cardinality(s)</code>	Tamaño de conjunto	integer

Regla 4 (SyntaxChecker): Funciones builtin usadas sin declarar -> **OK** (auto-resueltas).

Regla 8 (SyntaxChecker): Declarar builtin en "functions" -> **ERROR** (override no permitido).

Funciones User-Defined

Funciones personalizadas que **DEBEN declararse** en JSON "functions":

Declaración

```
{
  "name": "mifunc",
  "inputs": [
    {"name": "x", "type": "numeric", "domain": [0.0, 100.0]}
  ],
  "outputs": [
    {"name": "r", "type": "numeric", "domain": [0.0, 10.0]}
  ]
}
```

Verificación (FunctionChecker)

1. Buscar `mifunc.so` en search path
2. Si no existe, buscar `mifunc.o`
3. Si ninguno existe -> **ERROR**

Search path default: `./lib ./obj /usr/local/lib/csp`

Override search path:

`./FunctionChecker graph.json --path ./mylib:./lib:./obj`

Implementación en C

Signature:

```
void mifunc(double *inputs, int n_in, double *outputs, int n_out);
```

Ejemplo:

```
// mylib/mifunc.c
#include <math.h>

void mifunc(double *inputs, int n_in, double *outputs, int n_out) {
    if (n_in >= 1 && n_out >= 1) {
        outputs[0] = sqrt(inputs[0]); // r = sqrt(x)
    }
}
```

Compilar:

```
gcc -c -O2 -fPIC mylib/mifunc.c -o mylib/mifunc.o
```

Runtime

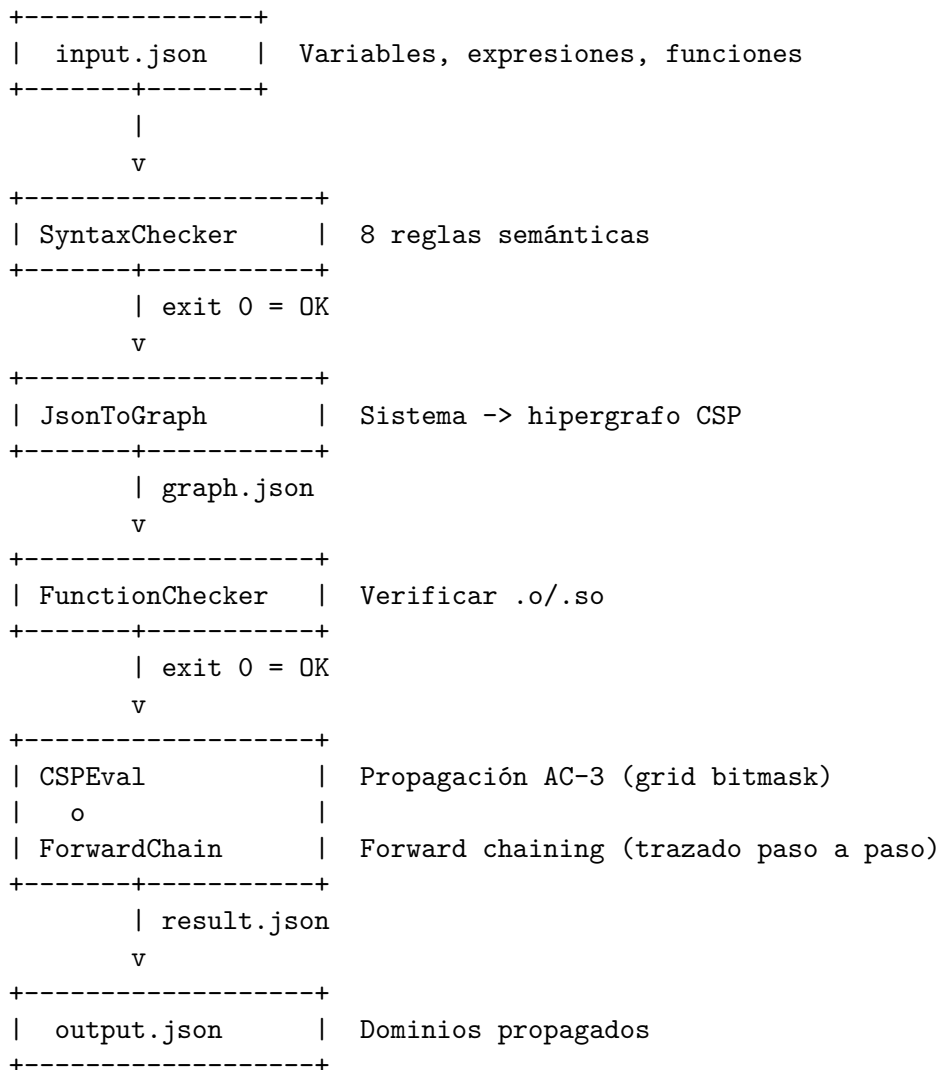
El motor (CSPEval, ForwardChain, etc.) carga la función con `dlopen`:

1. `handle = dlopen("mifunc.so", RTLD_NOW)`
2. `fptr = dlsym(handle, "mifunc")`
3. `fptr(inputs, n_in, outputs, n_out)`

IMPORTANTE: Funciones builtin NO aparecen en `func_refs` del grafo (resueltas internamente). Solo user-defined aparecen.

Arquitectura del Pipeline

Flujo de Datos



Programas del Pipeline

1. SyntaxChecker

Función: Validar sintaxis y semántica (8 reglas)

Entrada: system.json

Salida: Exit code 0 (OK) o 1 (error) + JSON con errores

8 Reglas:

1. Variables declaradas sin usar -> error
2. Funciones declaradas sin usar -> error
3. Variables usadas sin declarar -> error
4. Funciones usadas sin declarar -> error si no es builtin, OK si es builtin

5. Redeclaración de variable -> error
6. Redeclaración de función -> error
7. Tipo incorrecto en expresión -> error
8. Override de builtin -> error

Ejemplo:

```
./SyntaxChecker input.json
# Exit 0 = válido
# Exit 1 = errores de sintaxis
```

2. JsonToGraph

Función: Convertir sistema JSON a hipergrafo CSP

Entrada: system.json (validado por SyntaxChecker)

Salida: graph.json con: - nodes: Array de nodos AST - vars: Array de variables con IDs - constrs: Array de constraints con var_refs y func_refs - funcs: Array de funciones user-defined

Transformaciones: - Variables -> vars con id, name, kind (vkBoolean/vkSet/vkInteger/vkNumeric) - Expresiones -> AST nodes + constrs con referencias - Funciones user-defined -> funcs (builtins NO aparecen)

Ejemplo:

```
./JsonToGraph input.json > graph.json
```

3. FunctionChecker

Función: Verificar existencia de .o/.so para funciones user-defined

Entrada: graph.json

Salida: Exit code 0 (OK) / 1 (missing) / 2 (fatal)

Búsqueda: Para cada función en graph.json["funcs"]: 1. Buscar {funcname}.so en search path 2. Si no existe, buscar {funcname}.o 3. Si ninguno -> error

Ejemplo:

```
./FunctionChecker graph.json --path ./lib:./obj
# Exit 0 = todos los .o/.so encontrados
# Exit 1 = alguno falta
```

4. CSPEval

Función: Propagación de dominios AC-3 con grid bitmask

Entrada: graph.json

Salida: result.json con dominios propagados

Algoritmo: 1. Cargar grafo (vars, nodes, constraints) 2. Inicializar grid 20-pt para variables numeric 3. Iterar hasta punto fijo: - Evaluar cada constraint - Narrowing con grid bitmask (mg_narrow_*) - Actualizar dominios 4. Output: status CONSISTENT/FAILED + dominios finales

Grid bitmask 20-pt: - 21 puntos: $\text{GridPts}[k] = \text{Lo} + k * (\text{Hi} - \text{Lo}) / 20$ - Bitmask 32 bits:
bit $k = 1 \rightarrow$ punto activo - Narrowing: `mg_narrow_gt/ge/lt/le/iv(pts, mask, n, thresh)`

Ejemplo:

```
./CSPEval graph.json > result.json
```

5. ForwardChain

Función: Forward chaining con trazado paso a paso

Entrada: `graph.json`

Salida: JSON con trace de cada paso

```
{
  "steps": [
    {"step": 0, "constraint": 0, "action": "evaluate", "result": {...}},
    {"step": 1, "constraint": 1, "action": "narrow", "var": 0, ...}
  ],
  "final_state": {...}
}
```

Útil para: Debugging, visualización, auditoría de propagación.

Ejemplo:

```
./ForwardChain graph.json > trace.json
```

6. FwdConsistency

Función: AC-3 hacia adelante (verificación sin modificar dominios)

Salida: `{"consistent": true|false}`

7. BwdConsistency

Función: Proyección inversa por árbol de expresiones

Salida: JSON con dominios refinados

Compilación e Instalación

Requisitos

- **Free Pascal Compiler (fpc)** $\geq 3.2.0$
- **gcc** (C compiler)
- **make**

Instalación de Dependencias

Ubuntu/Debian

```
sudo apt install fpc gcc make
```

Fedora/RHEL

```
sudo dnf install fpc gcc make
```

Arch Linux

```
sudo pacman -S fpc gcc make
```

Compilación

```
cd /home/rodo/parser
```

```
make
```

Esto compila: 1. Bibliotecas C (minimath_*.o, roaring.o, minimath_grid.o) 2. 9 programas Pascal

Verificación

Listar ejecutables

```
ls -lh SyntaxChecker JsonToGraph FunctionChecker CSPEval ForwardChain
```

Ejecutar pipeline de ejemplo

```
./csp-pipeline.sh json/example.json
```

Limpieza

Limpiar archivos objeto

```
make clean
```

Output:

obj/.o*

obj/.ppu*

Ejecutables

Uso

Pipeline Completo

Script `csp-pipeline.sh` ejecuta todas las etapas:

```
./csp-pipeline.sh input.json
```

Secuencia: 1. SyntaxChecker input.json (valida) 2. JsonToGraph input.json > /tmp/graph.json (convierte) 3. FunctionChecker /tmp/graph.json -path ./lib:./obj (verifica .o/.so) 4. CSPEval /tmp/graph.json > /tmp/result.json (propaga) 5. Output: /tmp/result.json

Programas Individuales

Validar Sintaxis

```
./SyntaxChecker input.json
# Exit 0 = OK
# Exit 1 = errores -> JSON con lista de errores
```

Generar Hipergrafo

```
./JsonToGraph input.json > graph.json
```

Verificar Funciones

```
./FunctionChecker graph.json --path ./lib:./obj
# Exit 0 = todos los .o/.so encontrados
# Exit 1 = alguno falta
```

Propagar Dominios

```
# AC-3 con grid bitmask
./CSPEval graph.json > result.json

# Forward chaining con trace
./ForwardChain graph.json > trace.json

# Verificación AC-3 hacia adelante
./FwdConsistency graph.json

# Proyección inversa
./BwdConsistency graph.json
```

Formato JSON de Entrada

Estructura Básica

```
{
  "variables": [...],
  "expressions": [...],
```



```

    "functions": [...]
}

```

Variables

ARRAY (no dict). Cada elemento:

```

{
  "name": "temperatura",
  "type": "numeric",
  "domain": [0.0, 100.0],
  "value": [20.0, 25.0]
}

```

Tipos: - **boolean:** domain=[true, false], value=subconjunto - **set:** domain=["A", "B", ...], value=subconjunto - **integer:** domain=[1, 5, 10, ...], value=subconjunto - **numeric:** domain=[lo, hi], value=[lo', hi']

Expresiones

ARRAY de:

```

{
  "constraints": [
    "temperatura > 20.0",
    "presion IN {ALTA, MEDIA}",
    "activo AND (nivel > umbral)"
  ]
}

```

Funciones (Opcional)

Solo funciones user-defined (NO builtins):

```

{
  "name": "mifunc",
  "inputs": [
    {"name": "x", "type": "numeric", "domain": [0.0, 100.0]}
  ],
  "outputs": [
    {"name": "r", "type": "numeric", "domain": [0.0, 10.0]}
  ]
}

```

Ejemplo Completo

```

{
  "variables": [
    {"name": "x", "type": "numeric", "domain": [0.0, 10.0], "value": [0.0, 10.0]},
    {"name": "y", "type": "numeric", "domain": [0.0, 10.0], "value": [0.0, 10.0]},
    {"name": "activo", "type": "boolean", "domain": [true, false], "value": [true, false]}
  ]
}

```

```
],  
  "expressions": [  
    {"constraints": ["x + y <= 10.0"]},  
    {"constraints": ["activo IMPLICA (x > 5.0)"]}  
  ]  
}
```

Ejecutar:

`./csp-pipeline.sh example.json`

Resultado: Dominios propagados de x, y, activo.

Grid Bitmask (Variables Numeric)

Motivación

Variables `numeric` con dominio continuo `[lo, hi]` necesitan discretización para: - Narrowing eficiente - Propagación iterativa - Consistencia con AC-3

Solución: Grid 20-pt + bitmask de 32 bits.

Estructura

Nivel 1: Interval

```
Lo, Hi: Double; // Bounds del dominio
```

Nivel 2: Grid 20-pt

```
GridPts: array[0..20] of Double; // 21 puntos
GridStep: Double;                // (Hi - Lo) / 20
```

```
// Inicialización:
```

```
GridStep := (Hi - Lo) / 20.0;
for k := 0 to 20 do
  GridPts[k] := Lo + k * GridStep;
```

Nivel 3: Bitmask

```
GridMask: LongWord; // 32 bits (usa 21)
```

```
// Inicialización:
```

```
GridMask := $1FFFFFF; // 0x1FFFFFF = 21 bits activos
```

Bit k = 1 -> `GridPts[k]` está activo **Bit k = 0** -> `GridPts[k]` eliminado por narrowing

Operaciones de Narrowing

Funciones C en `minimath_grid.c`:

```
uint32_t mg_narrow_gt(double *pts, uint32_t mask, int n, double thresh);
uint32_t mg_narrow_ge(double *pts, uint32_t mask, int n, double thresh);
uint32_t mg_narrow_lt(double *pts, uint32_t mask, int n, double thresh);
uint32_t mg_narrow_le(double *pts, uint32_t mask, int n, double thresh);
uint32_t mg_narrow_iv(double *pts, uint32_t mask, int n, double lo, double hi);
```

Ejemplo: `x > 5.0`

```
// Antes: GridMask = 0x1FFFFFF (todos activos)
// Narrowing:
GridMask := mg_narrow_gt(GridPts, GridMask, 21, 5.0);
// Después: solo bits con GridPts[k] > 5.0 activos
```

Sincronización con Active[]

Después de narrowing, actualizar `Active[]`:

```
for k := 0 to 20 do
  Active[k] := (GridMask and (1 shl k)) <> 0;
```

Luego recalcular bounds:

```
RefreshIntegerBounds; // Actualiza Lo, Hi desde puntos activos
```

Ventajas

1. **Rapidez:** Operaciones bitwise son $O(1)$
2. **Precisión:** 20 puntos = resolución suficiente para mayoría de CSP
3. **Memoria:** 21 puntos + 1 int (96 bytes) vs. representación densa
4. **Composición:** Compatible con interval arithmetic (Lo, Hi)

Desventajas

1. **Límite de puntos:** Máximo 32 puntos (32 bits)
2. **Precisión fija:** No adaptativa
3. **Overhead:** Sincronización `Active[]` después de cada narrowing

Alternativas Consideradas

- **Interval arithmetic puro:** Sin discretización -> propagación débil
- **Malla adaptativa:** Complejidad alta
- **Roaring bitmaps:** Overhead para 20 puntos
- **Grid 100-pt:** Excede 32 bits, requiere múltiples words

Conclusión: Grid 20-pt + 32-bit mask = balance óptimo.

8 Reglas Semánticas (SyntaxChecker)

Regla 1: Variables Declaradas Sin Usar

Error: Variable declarada pero no aparece en ninguna expresión.

Ejemplo:

```
{
  "variables": [
    {"name": "x", "type": "numeric", "domain": [0.0, 10.0], "value": [0.0, 10.0]},
    {"name": "y", "type": "numeric", "domain": [0.0, 10.0], "value": [0.0, 10.0]}
  ],
  "expressions": [
    {"constraints": ["x > 5.0"]}
  ]
}
```

Error: Variable y declarada pero no usada.

Regla 2: Funciones Declaradas Sin Usar

Error: Función declarada en "functions" pero no aparece en ninguna expresión.

Ejemplo:

```
{
  "functions": [
    {"name": "mifunc", "inputs": [...], "outputs": [...]}
  ],
  "expressions": [
    {"constraints": ["x > 5.0"]}
  ]
}
```

Error: Función mifunc declarada pero no usada.

Regla 3: Variables Usadas Sin Declarar

Error: Variable usada en expresión sin declaración en "variables".

Ejemplo:

```
{
  "variables": [
    {"name": "x", "type": "numeric", "domain": [0.0, 10.0], "value": [0.0, 10.0]}
  ],
  "expressions": [
    {"constraints": ["x + y > 10.0"]}
  ]
}
```

Error: Variable y usada sin declarar.

Regla 4: Funciones Usadas Sin Declarar

Comportamiento: - Si función NO es builtin -> **ERROR** - Si función ES builtin -> **OK** (auto-resuelta)

Builtin list: abs, sqrt, sqr, sin, cos, tan, asin, acos, atan, exp, ln, log, pow, interval, complex, cardinality

Ejemplo OK:

```
{
  "expressions": [
    {"constraints": ["y = sqrt(x)"]}
  ]
}
```

sqrt es builtin -> OK sin declarar.

Ejemplo ERROR:

```
{
  "expressions": [
    {"constraints": ["y = mifunc(x)"]}
  ]
}
```

mifunc NO es builtin -> ERROR sin declarar.

Regla 5: Redeclaración de Variable

Error: Variable declarada más de una vez en "variables".

Ejemplo:

```
{
  "variables": [
    {"name": "x", "type": "numeric", "domain": [0.0, 10.0], "value": [0.0, 10.0]},
    {"name": "x", "type": "integer", "domain": [1, 10], "value": [1, 10]}
  ]
}
```

Error: Variable x declarada 2 veces.

Regla 6: Redeclaración de Función

Error: Función declarada más de una vez en "functions".

Ejemplo:

```
{
  "functions": [
    {"name": "mifunc", "inputs": [...], "outputs": [...]},
    {"name": "mifunc", "inputs": [...], "outputs": [...]}
  ]
}
```

Error: Función `mifunc` declarada 2 veces.

Regla 7: Tipo Incorrecto en Expresión

Error: Operación incompatible con tipo de variable.

Ejemplos:

- `x + y` donde `x` es `boolean` -> ERROR (suma requiere numeric/integer)
- `A AND B` donde `A` es `numeric` -> ERROR (AND requiere boolean)
- `estado > 10` donde `estado` es `set` -> ERROR (`>` requiere numeric/integer)

Regla 8: Override de Builtin

Error: Intentar redeclarar una función builtin en `"functions"`.

Ejemplo:

```
{  
  "functions": [  
    {"name": "sqrt", "inputs": [...], "outputs": [...]}  
  ]  
}
```

Error: `sqrt` es builtin, no se puede override.

Razón: Builtins están pre-registradas y optimizadas. User-defined functions NO pueden reemplazarlas.

Integración con GecodeInfo

Análisis del Espacio de Soluciones

Después de propagar dominios con CSPEval, se puede analizar el espacio de soluciones completo con **GecodeInfo** (ubicado en `/home/rodo/gecode/`).

Flujo de Trabajo

```
input.json
  |
  v
JsonToGraph -> graph.json
  |
  v
CSPEval -> result.json
  |
  v
GecodeInfo -> solutions.json
```

Uso

```
# 1. Pipeline CSP
./csp-pipeline.sh json/example.json > /tmp/graph.json

# 2. Análisis con Gecode
cd /home/rodo/gecode
./GecodeInfo /tmp/graph.json
```

Salida de GecodeInfo

```
{
  "total_solutions": 42,
  "analysis": [
    {
      "name": "x",
      "type": "numeric",
      "if_fixed": false,
      "values": [1.5, 2.3, 5.7]
    },
    {
      "name": "y",
      "type": "integer",
      "if_fixed": true,
      "value": 10
    }
  ]
}
```


Campos: - `total_solutions`: Número total de soluciones encontradas - `analysis`: Lista de variables con sus valores posibles - `if_fixed`: `true` si variable tiene valor único, `false` si múltiples

Bug Crítico Resuelto

Problema: Nodos `CT_INTERVAL` en AST usan campos `"lo"/"hi"` no `"left"/"right"`.

Síntoma: Modelo FAILED con intervalos `[0, 0]` incorrectos.

Solución en UCSPJson.pas (GecodeInfo):

```
// ParseNodeBlock - al final:
if Node.Left = -1 then begin
  P := FindKey(Blk, 'lo', 1);
  if P > 0 then Node.Left := ReadInt(Blk, P);
end;
if Node.Right = -1 then begin
  P := FindKey(Blk, 'hi', 1);
  if P > 0 then Node.Right := ReadInt(Blk, P);
end;
```

Esto parsea correctamente nodos `interval(lo, hi)`.

Variables Numéricas: Rescalado a Malla

GecodeInfo rescala variables numeric a ~20 puntos de malla para análisis:

```
procedure RescaleNumericVars;
const
  GRID_POINTS = 20;
var
  Lo, Hi, Step: Double;
  ...
begin
  Step := (Hi - Lo) / GRID_POINTS;
  // Discretizar dominio a GRID_POINTS valores
end;
```

Esto permite enumerar soluciones en espacio discreto.

Ejemplos

Ejemplo 1: Sistema Simple

Archivo: example1.json

```
{
  "variables": [
    {"name": "x", "type": "numeric", "domain": [0.0, 10.0], "value": [0.0, 10.0]},
    {"name": "y", "type": "numeric", "domain": [0.0, 10.0], "value": [0.0, 10.0]}
  ],
  "expressions": [
    {"constraints": ["x + y <= 10.0"]},
    {"constraints": ["x >= 2.0"]},
    {"constraints": ["y >= 3.0"]}
  ]
}
```

Ejecutar:

```
./csp-pipeline.sh example1.json
```

Resultado esperado: - Dominio de x: [2.0, 7.0] ($10 - 3 = 7$) - Dominio de y: [3.0, 8.0] ($10 - 2 = 8$)

Ejemplo 2: Conjuntos

Archivo: example2.json

```
{
  "variables": [
    {"name": "estado", "type": "set", "domain": ["ALTA", "MEDIA", "BAJA"], "value": ["ALTA", "MEDIA", "BAJA"]},
    {"name": "activo", "type": "boolean", "domain": [true, false], "value": [true, false]}
  ],
  "expressions": [
    {"constraints": ["estado IN {ALTA, MEDIA}"]},
    {"constraints": ["activo IMPLICA (estado = ALTA)"]}
  ]
}
```

Ejecutar:

```
./csp-pipeline.sh example2.json
```

Resultado esperado: - estado: ["ALTA", "MEDIA"] - Si activo = true -> estado = "ALTA"

Ejemplo 3: Función User-Defined

Archivo: example3.json

```
{
  "variables": [
    {"name": "x", "type": "numeric", "domain": [0.0, 100.0], "value": [16.0, 16.0]},
    {"name": "y", "type": "numeric", "domain": [0.0, 10.0], "value": [0.0, 10.0]}
  ]
}
```

```

],
"expressions": [
  {"constraints": ["y = mysqrt(x)"]}
],
"functions": [
  {
    "name": "mysqrt",
    "inputs": [{"name": "x", "type": "numeric", "domain": [0.0, 100.0]}],
    "outputs": [{"name": "r", "type": "numeric", "domain": [0.0, 10.0]}]
  }
]
}

```

Implementación C:

```

// lib/mysqrt.c
#include <math.h>

void mysqrt(double *inputs, int n_in, double *outputs, int n_out) {
  if (n_in >= 1 && n_out >= 1) {
    outputs[0] = sqrt(inputs[0]);
  }
}

```

Compilar:

```
gcc -c -O2 -fPIC lib/mysqrt.c -o lib/mysqrt.o
```

Ejecutar:

```

./FunctionChecker graph.json --path ./lib
./CSPEval graph.json

```

Resultado esperado: - y: [4.0, 4.0] ($\sqrt{16} = 4$)

Conclusiones

Ventajas del Sistema

1. **Modularidad:** 9 programas standalone, fácil de mantener
2. **Composición:** Pipeline vía JSON, extensible sin recompilar
3. **Eficiencia:** Grid bitmask 20-pt para narrowing rápido
4. **Precisión:** Interval arithmetic + discretización controlada
5. **Flexibilidad:** 4 tipos de variables + funciones user-defined
6. **Portabilidad:** Pascal + C estándar, sin dependencias complejas
7. **Debugging:** Salida JSON intermedia inspeccionable
8. **Extensibilidad:** Agregar etapas sin modificar código existente

Casos de Uso

- **CSP clásicos:** N-queens, graph coloring, scheduling
- **Sistemas de control:** Sensores, actuadores, restricciones de estado
- **Optimización:** Búsqueda de soluciones óptimas en espacio discreto
- **Verificación:** Chequeo de consistencia de especificaciones
- **Machine Learning:** Feature constraints, domain knowledge

Próximos Pasos

1. **Completar documentación** (02-04)
2. **Agregar tests de integración** (pipeline completo)
3. **Optimización de propagación** (skip constraints sin cambios)
4. **Soporte para arrays multidimensionales**
5. **Cache de resultados** (CSPEval)
6. **Generación de PDFs** con pandoc

Referencias

- **Free Pascal:** <https://www.freepascal.org/>
- **Constraint Programming:** Handbook of Constraint Programming (Rossi et al.)
- **AC-3 Algorithm:** Mackworth, “Consistency in Networks of Relations”, 1977
- **Interval Arithmetic:** Moore, “Interval Analysis”, 1966
- **Pratt Parser:** <https://matklad.github.io/2020/04/13/simple-but-powerful-pratt-parsing.html>

Fin del Manual de Introducción